

Digital Twin Project Report – PC Performance Twin

Digital Twins (WS 2025/26)

Montanuniversität Leoben

Students:

Harold Stiven Vargas Henao & Wilson Santiago Bonilla Guevara



Project Definition – Server Digital Twin

Purpose and Problem Statement

Modern computing systems, especially servers, operate under dynamic and unpredictable workloads. Understanding their behavior under varying resource demands is crucial for ensuring performance, reliability, and energy efficiency. However, conventional monitoring tools (such as Grafana or Prometheus) only provide descriptive information of the system in real time — they do not allow controlled experimentation or predictive modeling. The purpose of this project is to build a Digital Twin of a computing server that can simulate different workloads (CPU, memory, and disk stress), record real-time system responses, and generate datasets that serve as the foundation for predictive models of server performance.

Background and Gaps

Existing monitoring solutions and observability platforms focus mainly on data visualization and alerting. While they capture real-time data, they lack experimentation control and do not produce reproducible datasets for simulation or learning. Research on infrastructure twins and cloud performance modeling demonstrates the potential of combining telemetry with simulation-based experimentation, but these approaches often require complex cloud infrastructure, external APIs, or vendor-specific sensors. The gap lies in the lack of a lightweight, open-source framework that allows local data collection, experiment scheduling, and system behavior analysis without relying on third party monitoring suites

Contribution

This project contributes to a self-contained and reproducible Digital Twin architecture for a physical computing node (server or PC). Our contribution includes:

- A modular Python system that can generate, execute, and log resource-intensive workloads through “stressors” (CPU, memory, disk).
- A telemetry logger (`psutil_logger.py`) that records real system metrics in real time, forming a structured dataset(`system_metrics_dataset.csv`) for later analysis.
- A framework for creating controlled experiments that allow correlation between planned events and measured system responses.
- The foundation for future integration with predictive analytics or machine learning models, enabling simulation of "what-if" scenarios (e.g., predicting system overload).



Methods and Tools

The system consists of three layers:

1. **Stress Generation Layer**

cpu_stress.py, mem_stress.py and disk_stress.py simulate controlled loads on CPU, RAM, and disk.

2. **Experiment Control Layer**

make_schedule.py, runner.py and runner_parallel.py generate and execute an event schedule (schedule.json) and record experiment annotations (annotations.csv).

3. **Data Layer**

The psutil_logger.py module acts as a sensor, recording system metrics (CPU %, RAM %, I/O, network, processes, etc.) at defined intervals.

All scripts are implemented in Python using:

- psutil for system telemetry,
- subprocess and multiprocessing for load generation,
- csv and json for data storage, and
- matplotlib / seaborn for analysis and visualization.

The collected data is then cleaned, validated, and analyzed through Exploratory Data Analysis (EDA) techniques (heatmaps, scatter plots, and correlations).

Feasible Goals

The project focuses on the Digital Twin of a single server (asset-level twin), with emphasis on data generation, synchronization, and behavioral analysis.

Real-time feedback control or predictive automation is beyond current scope, but the dataset and architecture are designed to enable these in future work.

Output – Deliverables and Outcomes

Deliverables:

- system_metrics_dataset.csv – dataset containing real-time system metrics during controlled stress tests.
- annotations.csv – experiment timeline with start and end times of each stress event.
- schedule.json – automatically generated experiment plan.
- Project documentation and short report (data description, cleaning, preparation, and EDA).

Expected Outcomes:

- A validated experimental framework for data-driven server monitoring.
- A reproducible dataset suitable for predictive modeling.



- A conceptual and practical demonstration of a Digital Twin for computing infrastructure, aligning with the “Descriptive → Predictive” twin maturity model.

Physical System Definition

1. What system are you twinning?

The physical system is an Operational computational asset, understood as a set of hardware and software components that generate real-time status data that is operating under real-world workloads. It executes code, manages resources, handles storage operations, and communicates over networks. Its behavior changes dynamically based on user activity, process scheduling, and system-level resource allocation.

In this case, the physical system corresponds to a personal computing environment, whose internal and external variables evolve continuously during its operation.

2. What does the system do?

The PC transforms computational tasks into results:

- Executes processes
- Allocates CPU and RAM
- Reads/writes data to storage
- Sends/receives network data
- Manages background services

Its function is to deliver computational performance under varying loads.

3. How is it measured?

Measurement occurs via software sensors (telemetry):

- CPU usage (% and core count)
- RAM usage and availability
- Disk utilization and I/O throughput
- Network throughput (sent/received MB/s)
- Battery level (if applicable)
- Running process count

These signals represent internal operational states of the system.

4. What physical principles influence it?

Even though the PC is not a mechanical system, it follows:

- Load distribution across CPU cores



- Memory allocation, caching, paging
- Disk I/O queueing effects
- Network bandwidth limitations
- Thermal throttling (indirectly observable)

Inputs, States, Outputs Mapping

Inputs (u):

Factors influencing the PC:

- Applications opened
- CPU-intensive tasks
- Disk operations (copy, compress, install)
- Network transfers
- RAM pressure from multitasking
- OS background operations

Internal State (x):

Hidden computational states:

- CPU internal scheduling
- Memory fragmentation / caching
- Disk queue length and latency
- Network socket load
- Process scheduling load

Outputs (y):

Observable measurements:

- CPU%, RAM%, Disk%
- Disk I/O rates
- Network Throughput
- Predicted future load
- Anomaly scores
-

Behavior Rules and Constraints

Behavior Rules:

- CPU load increases CPU% until saturation.
- RAM pressure leads to paging and performance degradation.
- Disk I/O increases when large files or installations occur.
- Network load increases during downloads or streaming.
- Combined loads can produce nonlinear slowdowns.



Operational Constraints:

- CPU cannot exceed 100%.
- RAM must not reach 100% to avoid freezes.
- Disk saturation (>95%) dramatically increases latency.
- Network capacity is finite and influenced by external servers.

Dynamics:

- CPU reacts instantly to workload.
- RAM reacts more slowly.
- Disk exhibits lag due to buffering and queueing.
- Network varies with external conditions, Routes and traffic fluctuation.

Digital Twin Architecture

1. Physical System → PC
2. Sensors → psutil metrics
3. Preprocessing → cleaning, validation, resampling
4. Feature Engineering → trends, deltas, moving averages
5. Model → forecasting + anomaly detection
6. Behavior Rules → operational constraints
7. Outputs → predictions, alerts, performance indices
8. Update Loop → continuous real ↔ twin alignment

Data & Dataset Description

Measurements are stored in a structured CSV file, where each row corresponds to a temporary sampling instant. Each record contains the following variables:

Timestamp (datetime, UTC): Sampling time mark, standardized in ISO-8601 format.

CPU Metrics

- Cpu_percent (%): total CPU usage.
- Cpu_count_logical (int): number of logical cores.
- Cpu_count_physical (int): number of physical cores.

Memory Metrics

- ram_total_GB (float): total RAM capacity.
- ram_available_GB (float): available memory.
- Ram_used_percent (%): percentage of RAM used.
- Swap_used_percent (%): use of swap memory.

Disk Metrics

- disk_total_GB (float): total disk capacity.



- `disk_used_percent (%)`: percentage of use.
- `disk_read_MB (float)`: data read from the disk during the interval.
- `disk_write_MB (float)`: data written during the interval.

Network Metrics

- `net_sent_MB (float)`: volume sent in the interval.
- `net_recv_MB (float)`: volume received in the interval.

Battery Metrics

- `Battery_percent (%)`: battery level (can be 0 or NaN on desktops).
- `Battery_plugged (bool)`: indicates if the equipment is connected to power.

Process Metrics

- `Process_count (int)`: total number of active processes in the system.

Exploratory Data Analysis (EDA)

The exploratory data analysis corresponding to this dataset is available in a public GitHub github.com/haroldvahe/DigitalTwin. The entire process of initial inspection, visualization, statistical analysis, detection of atypical values and temporal behavior of the metrics is documented in the `System_Metrics_Analysis.ipynb` notebook, which contains the complete EDA made on the CSV file. This notebook allows you to reproduce in full the preliminary analysis that supports the construction of the digital twin.

Dataset overview and summary statistics

The dataset consists of approximately 62,000 time-stamped observations of system-level resource metrics, including CPU utilization, RAM usage, disk usage, disk I/O, network activity, and process count. Summary statistics indicate low average CPU load, moderate and stable memory usage, and near-constant disk utilization, suggesting a system operating mostly under normal conditions.

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
<code>cpu_percent</code>	62233.0	NaN	NaN	NaN	25.887277	18.025776	0.0	12.0	14.1	42.4	99.5
<code>cpu_count_logical</code>	62233.0	<NA>	<NA>	<NA>	32.0	0.0	32.0	32.0	32.0	32.0	32.0
<code>cpu_count_physical</code>	62233.0	<NA>	<NA>	<NA>	24.0	0.0	24.0	24.0	24.0	24.0	24.0
<code>ram_total_GB</code>	62233.0	NaN	NaN	NaN	31.71	0.0	31.71	31.71	31.71	31.71	31.71
<code>ram_used_percent</code>	62233.0	NaN	NaN	NaN	49.006598	4.725744	42.8	47.1	48.0	49.2	85.1
<code>ram_available_GB</code>	62233.0	NaN	NaN	NaN	16.169076	1.498468	4.72	16.1	16.49	16.78	18.12
<code>swap_used_percent</code>	62233.0	NaN	NaN	NaN	0.287115	0.033504	0.2	0.3	0.3	0.3	0.3
<code>disk_total_GB</code>	62233.0	NaN	NaN	NaN	930.5	0.0	930.5	930.5	930.5	930.5	930.5
<code>disk_used_percent</code>	62233.0	NaN	NaN	NaN	64.600201	0.207047	64.3	64.5	64.5	64.9	65.0
<code>disk_read_MB</code>	62233.0	NaN	NaN	NaN	0.175519	3.850356	0.0	0.0	0.0	0.0	472.937
<code>disk_write_MB</code>	62233.0	NaN	NaN	NaN	1.433766	10.616192	0.0	0.105	0.197	0.367	1363.55
<code>net_sent_MB</code>	62233.0	NaN	NaN	NaN	1.074591	1.909196	0.0	0.002	0.028	1.381	11.822
<code>net_recv_MB</code>	62233.0	NaN	NaN	NaN	0.170015	0.674747	0.0	0.006	0.016	0.051	11.916
<code>battery_percent</code>	62233.0	NaN	NaN	NaN	100.0	0.0	100.0	100.0	100.0	100.0	100.0
<code>battery_plugged</code>	62233	1	True	62233	NaN	NaN	NaN	NaN	NaN	NaN	NaN
<code>process_count</code>	62233.0	<NA>	<NA>	<NA>	347.214195	11.016905	331.0	340.0	344.0	352.0	437.0
<code>disk_io_total_MB</code>	62233.0	NaN	NaN	NaN	1.609285	11.459387	0.0	0.109	0.203	0.392	1363.652
<code>net_io_total_MB</code>	62233.0	NaN	NaN	NaN	1.244606	2.054422	0.0	0.008	0.248	1.599	14.302

Table 1 Dataset Description



Time-series behavior of system resources

Time-series analysis shows that CPU utilization fluctuates strongly over time, with frequent short-lived peaks reaching high values, while RAM usage remains comparatively stable with occasional spikes. This indicates bursty computational workloads combined with more persistent memory allocation patterns.

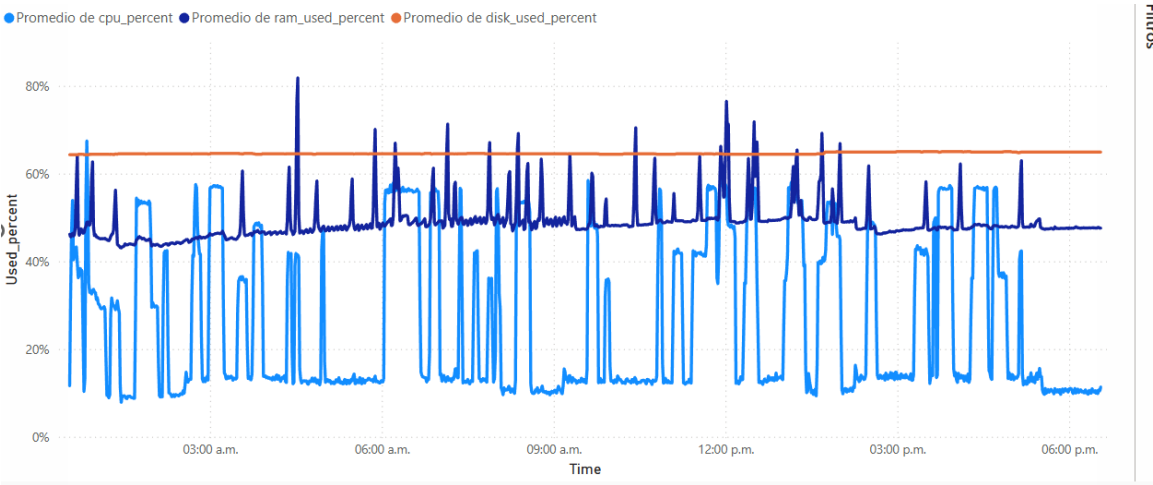


Figure 1 System Resource Utilization Over Time

Distribution analysis and operating regimes

The distributions reveal distinct operating regimes. CPU usage is right-skewed, dominated by low-load operation with rare high peaks. RAM usage exhibits a slightly bimodal distribution, suggesting the presence of discrete memory usage states. Swap usage shows clear bimodality, indicating transitions between memory pressure and normal operation. Disk usage remains nearly constant, reflecting a static disk allocation.

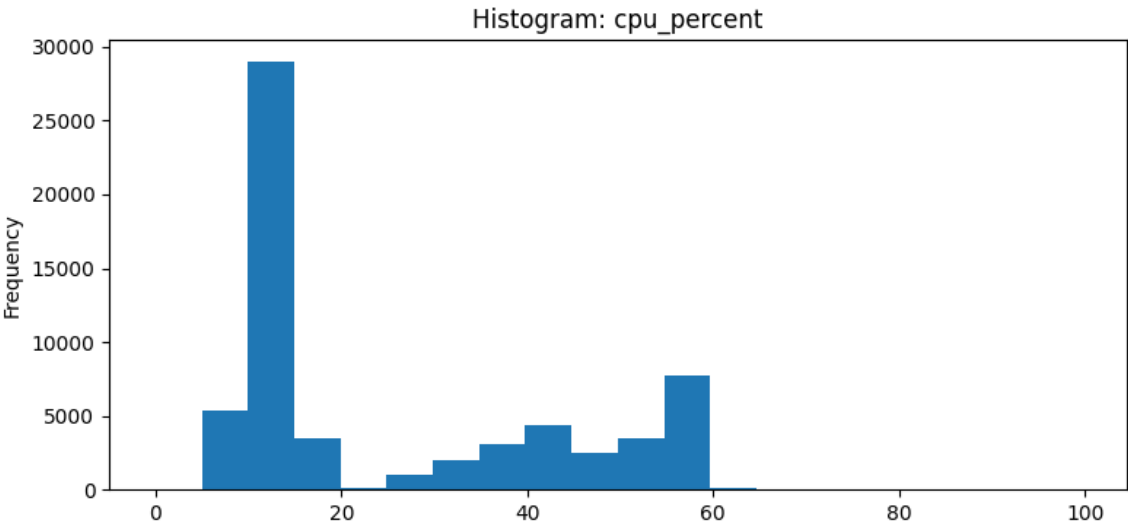


Figure 2 Histogram CPU Usage



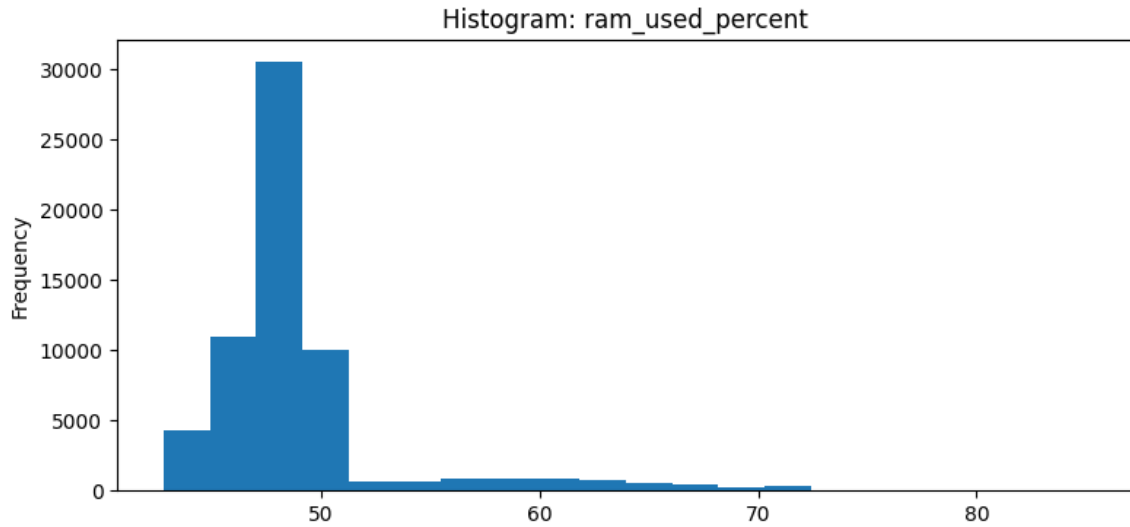


Figure 3 Histogram Ram Usage

Correlation and dependency analysis

Correlation analysis shows strong negative correlation between RAM used and available memory, as expected, while CPU usage exhibits weak correlation with other subsystems. This indicates a decoupled system architecture, where CPU-intensive tasks do not necessarily drive memory, disk, or network activity.

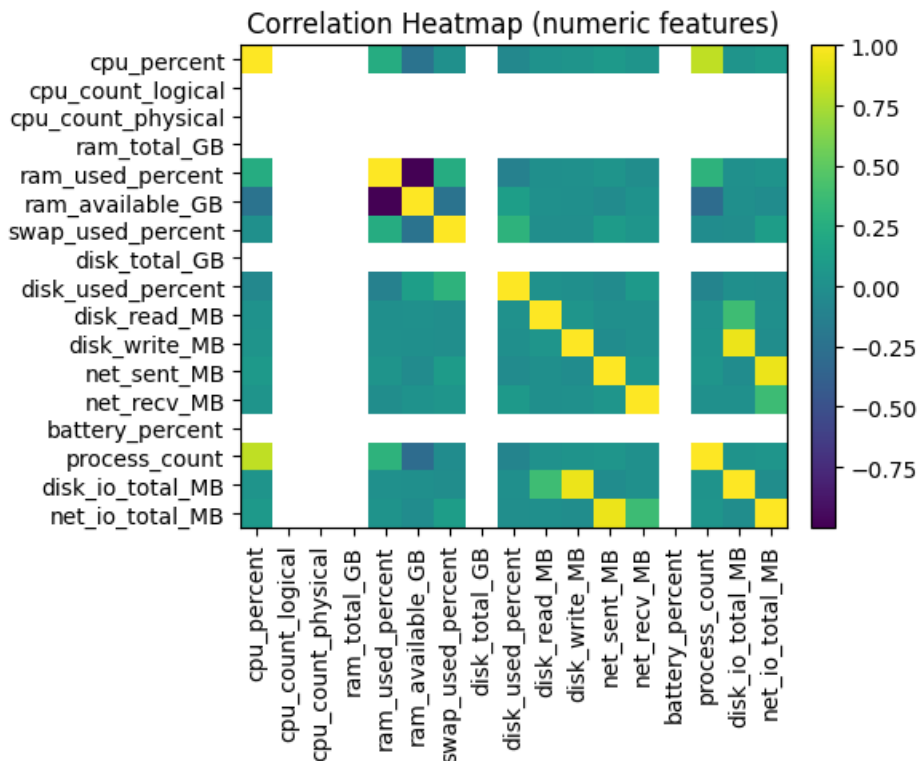


Figure 4 Correlation Heatmap

Model Placement

Model sits in:

Inputs (u) → Model → Outputs (y)

The model predicts:

- future CPU load
- risk of resource saturation
- anomalies in usage patterns

Relevant variables:

- CPU%, RAM%, Disk%, I/O, Network

Less relevant:

- Battery (if always plugged), process names

What the Model Represents and Why

Represents dynamic computational performance.

It is essential for:

- anticipating overload
- improving responsiveness
- detecting anomalies
- creating a predictive virtual replica

Update Loop Description

1. New telemetry arrives
2. Data cleaned and validated
3. Model predicts values
4. Behavior rules applied
5. Twin internal states updated
6. Comparison with real system
7. Deviations logged for anomaly detection

Limitations & Risks

The digital twin has several limitations inherent in the process of capturing, modeling and synchronizing metrics of the physical system. These limitations should be considered to properly interpret the results and ensure safe use of the model.

1. Scalability: The current approach is designed for a single physical system and may not scale efficiently to multiple machines or distributed environments.
 - i. The volume of data increases linearly with time.



- ii. The sampling frequency directly affects the size of the dataset and storage requirements.
 - iii. Training more complex models in growing datasets can involve long time and greater computational resources.
- 2. Sensor Noise & Measurement Variability: Metrics obtained through psutil may contain noise, rapid fluctuations, or inconsistencies, especially at short intervals.
 - i. Some values can be erratic due to the operating system scheduler.
 - ii. Metrics such as `cpu_percent` or `net_sent_MB` vary abruptly by nature.
 - iii. Failures in sensor reading can generate NaN values.

These conditions can affect the stability of predictive models and the detection of anomalies.

- 3. Model Drift: The system's load patterns can change over time, causing drift between the actual behavior and the twin model.
 - i. Installation of new applications.
 - ii. Changes in usage habits.
 - iii. Operating system or hardware updates.
 - iv. Variation in external workloads.

If the digital twin is not recalibrated periodically, its accuracy may deteriorate.

Description of the Chosen Models and Analytical Methods

This project adopts a **data-driven statistical and logical modeling approach** to represent the operational behavior of a computing server within a Digital Twin framework.

First, a **statistical baseline model** is constructed using descriptive statistics learned directly from the collected telemetry data. For key system metrics such as CPU usage, RAM usage, and swap utilization, the baseline defines a normal operating region based on the mean and standard deviation of the observed data. This baseline provides a quantitative reference for identifying deviations from typical system behavior.

Second, a **rule-based system state model** is introduced to transform continuous telemetry signals into discrete operational states. Based on predefined thresholds informed by both the statistical baseline and system-level intuition, the server is classified into a finite set of states, including *NORMAL*, *HIGH_CPU*, *HIGH_RAM*, *MEMORY_PRESSURE*, and *CRITICAL*. This abstraction enables an interpretable and structured representation of system behavior, which is well suited for Digital Twin applications.

Finally, a **state-transition predictive model** based on a discrete-time Markov chain is learned from the sequence of inferred system states. This model captures the probabilistic dynamics of state



transitions and enables estimation of the likelihood of entering higher-stress or critical operating conditions in the next step.

Together, these models form a coherent modeling pipeline that moves from continuous data to interpretable states and probabilistic behavioral prediction.

Justification of the Chosen Approach

A **data-driven modeling strategy** was selected due to the absence of explicit physical equations governing the internal dynamics of operating systems and resource schedulers. While physics-based models are effective for systems with well-defined governing laws, such formulations are not readily applicable to general-purpose computing infrastructure.

Machine learning models such as supervised regression or deep learning were intentionally not employed, as the dataset does not contain labeled failure events or explicit target variables. Additionally, the project prioritizes **interpretability, reproducibility, and conceptual clarity**, which are essential characteristics for a foundational Digital Twin.

The combination of **statistical modeling and rule-based logic** provides an effective balance between expressiveness and simplicity. This approach is fully data-driven, computationally efficient, and transparent, making it suitable for early-stage Digital Twin development. The Markov-based state-transition model extends this framework with a lightweight predictive capability, allowing probabilistic reasoning about short-term system evolution without introducing black-box behavior.

Evaluation Metrics and Validation Approach

The proposed models are evaluated using both quantitative metrics and qualitative validation techniques appropriate for state-based and probabilistic models.

For the statistical baseline model, validation is performed by measuring the proportion of observations that fall within the defined normal operating bounds. High coverage indicates that the baseline accurately captures typical system behavior.

The rule-based state model is evaluated through state distribution analysis, quantifying the proportion of time the system operates in each state. This provides insight into system stability and workload characteristics.

For the Markov state-transition model, evaluation focuses on:



- Transition probability analysis, particularly the persistence of states (diagonal dominance of the transition matrix).
- One-step state prediction consistency, measured as the agreement between the most probable predicted next state and the observed next state.
- Normalized confusion matrices to assess how well the model captures state persistence and transitions.

These metrics are used as **consistency and behavioral validation measures**, rather than traditional machine learning performance metrics.

Visual and Quantitative Assessment of Results

Visual validation plays a central role in assessing the modeling results. Time-series plots combining raw telemetry with statistical baselines demonstrate that most observations lie within normal operating bounds, while short-lived excursions correspond to periods of increased load.

System state timelines illustrate how continuous resource metrics are translated into discrete operational states over time. These plots show a clear predominance of the *NORMAL* state, with brief and well-defined transitions into higher-stress states.

Additional validation is provided through boxplots of CPU and RAM usage conditioned on system state. These visualizations confirm that each inferred state corresponds to a distinct and meaningful resource usage profile, indicating that the state definitions effectively separate different operational regimes.

Finally, heatmaps of the Markov transition matrix and normalized confusion matrices reveal strong state persistence, particularly for the *NORMAL* and *HIGH_RAM* states, while highlighting the transient nature of *HIGH_CPU* events. These results are consistent with the exploratory data analysis and confirm that the learned model captures meaningful behavioral dynamics.

Limitations and Failure Cases

Despite its effectiveness, the proposed modeling approach has several limitations. The statistical baseline assumes stationarity of system behavior and may not adapt to long-term changes in workload patterns or configuration drift. Thresholds used in the rule-based state model are system-specific and require recalibration when applied to different hardware or operating environments.

The Markov state-transition model assumes the Markov property, meaning that future states depend only on the current state and not on longer historical context. As a result, long-term dependencies and rare transition events may be underestimated, particularly for critical states that occur infrequently.

Finally, the model focuses on state detection and short-term probabilistic prediction rather than explicit forecasting of future resource values. While this is appropriate for a foundational Digital Twin, more advanced predictive capabilities would require time-series forecasting or supervised learning approaches, which are left for future work.

Digital Twin Integration and Reflection on the Results

The developed Digital Twin primarily serves a diagnostic purpose by identifying system states and stress conditions. Additionally, the Markov-based state transition model provides limited predictive capabilities by estimating the likelihood of future state transitions. Prescriptive decision-making is considered out of scope for this project.

How the model and analysis fit into a Digital Twin architecture

The developed Digital Twin integrates data acquisition, data analysis, and behavioral modeling to represent the operational state of a server-like computing system. The analytical core of the Digital Twin is composed of a statistical baseline model, a rule-based system state classifier, and a Markov state transition model.

Raw telemetry data collected from the physical system are continuously ingested and preprocessed. The cleaned and prepared data feed the analytical layer, where baseline statistics define normal operating ranges, rule-based logic assigns interpretable system states, and the Markov model captures temporal dependencies between states. Together, these components allow the Digital Twin to mirror both instantaneous conditions and dynamic behavior of the physical system.

Inputs to the Digital Twin

The Digital Twin operates on multiple categories of inputs:

- **Data streams:** High-frequency time-series telemetry, including CPU utilization, RAM usage, disk usage, disk I/O, network activity, and process count, collected using a psutil-based monitoring script.
- **Historical data:** Previously recorded system metrics used to compute statistical baselines and transition probabilities.
- **Metadata:** System configuration parameters such as logical and physical CPU cores, total memory capacity, disk capacity, and battery status indicators.

These inputs jointly define the observable state of the physical system and provide the necessary context for interpretation.

Outputs of the Digital Twin

The Digital Twin generates multiple outputs that support monitoring and decision making:



- **System states:** Discrete operational states such as NORMAL, HIGH_CPU, HIGH_RAM, MEMORY_PRESSURE, and CRITICAL, derived from rule-based classification.
- **Indicators:** Statistical indicators describing how often the system operates within normal bounds versus stress conditions.
- **Behavioral dynamics:** State transition probabilities derived from the Markov model, describing how the system evolves over time.
- **Decision support signals:** Identification of high-risk transitions that may lead to critical states.

Representation of results and visualization

The results are represented through multiple complementary visualizations. Time-series plots show the evolution of CPU, RAM, disk, and network metrics. Histograms and correlation heatmaps support the identification of stable operating regions and variable dependencies. State timelines overlay inferred system states on resource utilization curves, providing an interpretable view of system behavior over time. Transition matrices and state timelines visually communicate long-term behavioral patterns and stability characteristics.

These visualizations collectively enable monitoring, analysis, and decision support within the Digital Twin framework.

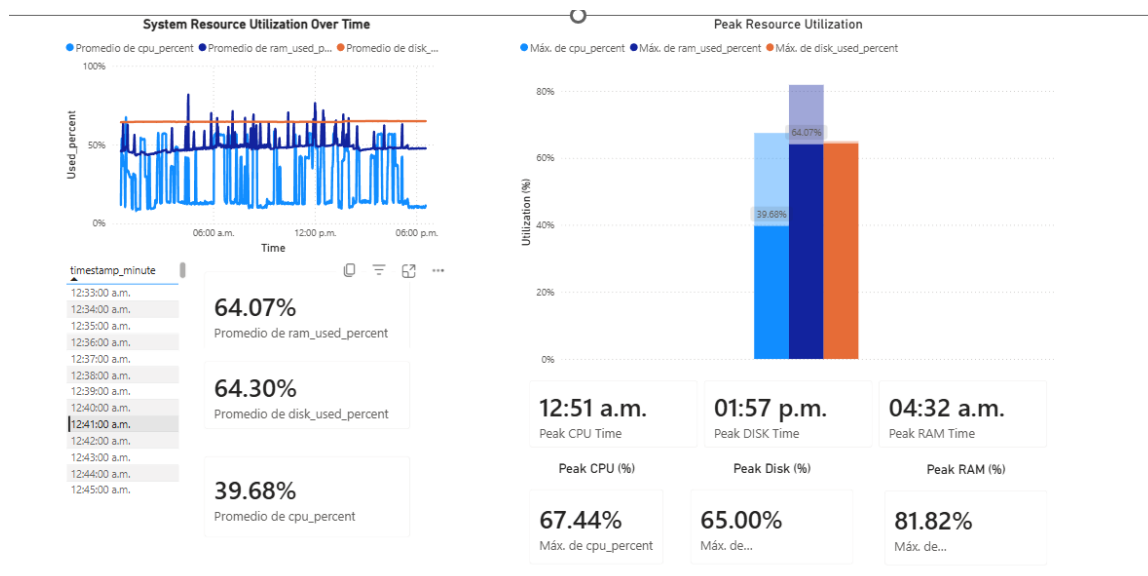


Figure 5 Digital Twin Monitoring Dashboard (Power BI Prototype)

The figure presents a prototype monitoring dashboard implemented in Power BI. The dashboard represents the visualization and decision-support layer of the Digital Twin. It integrates time-series resource utilization, aggregated indicators, and peak usage information to provide an intuitive overview of system behavior.

The dashboard enables users to monitor current and historical system states, identify periods of high resource utilization, and quickly detect potential stress conditions. While detailed analysis and modeling are performed offline, this visualization demonstrates how Digital Twin outputs can be operationalized and consumed by end users.

Key lessons learned

The analysis demonstrates that server-like systems exhibit well-defined operational states and strong temporal persistence. Most system behavior remains within normal operating bounds, with stress conditions appearing as short-lived events rather than sustained failures. The combination of statistical baselines and rule-based logic proved effective for producing interpretable and defensible Digital Twin outputs.

What worked well and what did not

What worked well was the clarity and interpretability of the rule-based system state model and the stability captured by the Markov transition model. These approaches align well with Digital Twin requirements, where explainability is critical. However, the approach is limited in its ability to anticipate entirely novel system behaviors or external disturbances not present in historical data.

Possible improvements and next steps

Future improvements could include integrating predictive forecasting models, extending the state space to incorporate additional system components, and enabling real-time deployment with live data ingestion. Root-cause analysis and adaptive thresholding could further enhance the Digital Twin's decision-support capabilities.

